

A Systematic Approach to Feature Reduction and Encoding for Efficient Text Classification

Janakiram Suhas Goli
University College Cork, Ireland

Abstract: This project presents a structured and scalable feature engineering pipeline designed for classification tasks involving both unstructured textual data and structured categorical variables. The workflow begins with comprehensive Natural Language Processing (NLP) techniques, including tokenization, normalization, and vectorization, to transform raw text into analyzable formats. To address the high dimensionality inherent in textual features, the Chi-Squared (χ^2) statistical test is employed for feature selection, reducing the feature space to the top 2,000–3,000 most relevant terms based on their predictive power. Simultaneously, categorical features are encoded using Ordinal Encoding, ensuring compatibility with downstream machine learning models. The combined feature set is then further refined through a Best-K feature selection strategy, optimizing the input space for model training. Finally, a Logistic Regression classifier is applied to the selected features to perform the predictive task. This pipeline demonstrates a balanced approach to dimensionality reduction, interpretability, and model performance, making it well-suited for various applications in text-driven predictive analytics.

I Introduction

With the massive growth of digital content online, text classification has become an important task in Natural Language Processing (NLP). It helps in automatically sorting and labeling text for applications like spam detection, sentiment analysis, and topic categorization. However, working with text data is not easy. It is often unstructured, very large in size, and can include too many irrelevant features. These challenges make it harder for machine learning models to perform well unless the data is carefully processed and simplified.

This project introduces a step-by-step pipeline designed to improve text classification by focusing on strong feature engineering. The first step is to clean and prepare the text using NLP techniques such as tokenization, removing stop words, normalization, and vectorization. These methods turn raw text into a format that machine learning algorithms can understand. Similar approaches were followed in previous research, where techniques like Bag-of-Words and TF-IDF helped capture important patterns from text [1].

In the second step, we reduce the number of text features using the Chi-Squared (χ^2) statistical test. This method helps us keep only the top 2,000–3,000 most relevant words that are most useful for making predictions. This idea is supported by earlier studies, which showed that reducing feature size can improve both the speed and accuracy of models [1], [2].

Next, we handle any other data in the dataset that is in the form of categories, such as labels or types—by converting them into numbers using Ordinal Encoding. This makes sure all the data, whether text or categorical, is in a format that can be used together by the machine learning model.

In the fourth step, we apply a feature selection method called Best-K to pick the most important features from the combined dataset. This helps in focusing the model only on the most useful information, improving performance and avoiding overfitting.

Finally, we use Logistic Regression (LR) to train the model. Although many studies, including those by Ibrahim et al. and Dong et al., explore more complex ensemble methods like Adaboost and LightGBM [1], [2], our approach shows that even a simple model like LR can give strong results when supported by a carefully designed preprocessing and feature selection pipeline.

II Literature Review

Text classification remains a fundamental problem in Natural Language Processing (NLP), with wide-ranging applications such as spam detection, sentiment analysis, and document categorization. Recent research has increasingly explored the use of ensemble learning techniques to enhance classification performance, particularly in handling challenges such as high-dimensional feature spaces and class imbalance. The two key studies reviewed here by Yusuf et al. [1] and Dong and Han [2] offer valuable insights into the impact of ensemble methods and highlight opportunities for more structured and effective feature engineering pipelines.

In their study, Yusuf et al. [1] conducted a comprehensive experimental evaluation comparing traditional supervised learning models with various ensemble techniques for text classification. Using the YouTube Spam Collection dataset, the authors applied standard NLP preprocessing methods, including stop-word removal, punctuation elimination, and stemming, to clean the text data. Feature extraction was performed using Bag-of-Words (BoW) and Term Frequency–Inverse Document Frequency (TF-IDF) methods, both widely used approaches for converting text into numerical feature vectors.

The processed features were input into a series of classifiers, ranging from base learners such as Logistic Regression (LR), Naïve Bayes (NB), and Support Vector Machines (SVM), to ensemble techniques including Random Forest (RF), Adaboost, Gradient Boosting (GBM), and LightGBM. The study found that ensemble models consistently outperformed their base counterparts. In particular, Adaboost and LightGBM achieved superior classification accuracy and stability across 5-fold and 10-fold cross-validation experiments. These findings highlight the potential of ensemble learning in improving predictive performance in text classification tasks, especially when dealing with high-dimensional, noisy, or imbalanced datasets.

Complementing this, Dong and Han [2] investigated the comparative effectiveness of several ensemble methods applied to Naïve Bayes and SVM classifiers. Recognizing that text data typically includes a vast number of sparse and redundant features, they explored six homogeneous ensemble strategies, such as k-fold partitioning, bagging, boosting, and biased sampling, as well as a heterogeneous ensemble that combined Naïve Bayes and SVM outputs using a neural network as a meta-classifier. Their results revealed that boosting significantly enhanced the performance of Naïve Bayes models, particularly in the presence of class imbalance. However, the same technique yielded reduced effectiveness when applied to SVMs, suggesting that the benefits of ensemble

learning are highly dependent on the choice of base model.

While both studies demonstrate the strengths of ensemble learning, they also expose several limitations. Neither study employed explicit statistical feature selection techniques, such as the Chi-Squared (χ^2) test, to reduce the dimensionality of the text features before training. Additionally, categorical (non-textual) data—often present in real-world datasets—was not incorporated or encoded, limiting the models’ applicability in hybrid data environments.

This gap presents an opportunity for improved classification pipelines. Building on the foundation laid by these studies, the present work introduces a structured four-stage approach: beginning with advanced NLP preprocessing, followed by Chi-Squared-based feature selection to retain the most relevant 2,000–3,000 features; applying Ordinal Encoding to convert structured categorical data; and concluding with a Best-K feature selection strategy to refine the combined feature set before classification using Logistic Regression. This approach addresses the dimensionality and integration limitations identified in prior literature, while demonstrating that strong performance can be achieved with simpler models when supported by effective feature engineering.

III Methodology

The goal of this project was to predict a hidden column (top category, bottom category) from a dataset stored in Parquet format, which included 25,514 rows and 15 columns. The prediction was based primarily on the title column, which contains short text entries. To achieve this, we applied a combination of Natural Language Processing (NLP) techniques and machine learning models, focusing on building a reliable pipeline for text classification.

According to the pipeline illustrated in Figure ??, the complete process is broken down into nine key stages.

This step-by-step workflow ensures that raw text data is cleaned, transformed into a numerical format, and then used to train and evaluate machine learning models consistently and efficiently.

III.I Data Preprocessing

To prepare the text for modeling, several NLP preprocessing steps were applied to clean and normalize the data:

- **Lowercasing:** All text was converted to lowercase.

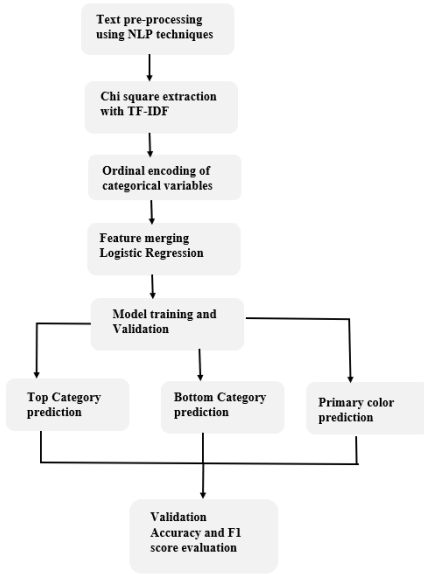


Figure 1: Text Classification Methodology

- **Punctuation Removal:** Symbols and punctuation were stripped out using regular expressions.
- **Tokenization:** Text was split into individual words using NLTK.
- **Stop Word Removal:** Common, less meaningful words were removed.
- **Lemmatization:** Words were converted to their base form (e.g., “running” → “run”).

The processed output was stored in a new column, pre-processed title, and used in the next steps.

III.II Text Feature Extraction with TF-IDF

Preprocessed text is converted into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization. TF-IDF emphasizes rare but important words, helping to differentiate between product types. For computational efficiency, the number of features is limited to using maximum features.

III.III Feature Selection

To focus only on the most important words, Chi-Squared (X^2) feature selection was used. The TF-IDF matrix was analyzed to identify and retain the top 2,000 features that had the strongest relationship with the target labels. This reduced dimensionality while keeping informative terms.

III.IV Ordinal Encoding of Categorical Variables

Structured categorical fields—such as material, occasion, style, pattern, etc, are transformed using Ordinal Encoding. Each unique category is assigned a numerical value, making them compatible with scikit-learn models. This allows seamless integration with text-based features.

III.V Feature Merging

The selected TF-IDF features are horizontally concatenated with the encoded categorical variables using sparse matrix operations (hstack). This forms a unified feature matrix (X_{final}) used for model training.

III.VI Model Training and Validation

The finalized feature matrix, comprising both text-derived and categorical components, is used to train multiple independent classifiers, each targeting a specific prediction task. These include the top-level category, bottom-level category of the item. Each model is trained using the same consistent set of optimized features selected through Chi-Squared analysis and combined with encoded categorical data.

- **Top Category Prediction :** A Logistic Regression classifier is employed to predict the topcategoryid. The dataset is partitioned into an 80/20 split for training and validation, respectively. The model is trained using the liblinear solver, which is well-suited for high-dimensional, sparse feature spaces. Various solvers were also used, but liblinear outperformed those solvers and gave the best results. The reason why LR was used because since the top category had 12-15 classes it performed well, whereas the bottom category had 2900 classes which were difficult to train the model. Performance is evaluated using accuracy and the weighted F1 score, ensuring the evaluation accounts for potential class imbalances.
- **Bottom Category Prediction :** A separate instance of Random Forest Classifier is trained to predict the bottomcategoryid, leveraging the same input feature matrix. Identical preprocessing, model parameters, and evaluation metrics are used, allowing for consistent comparison between the top and bottom-level classification tasks. The reason of choosing RF is because of the huge number of classes which does not go well with the simple Logistic Regression.

This multi-target modeling approach allows for specialized learning per task, enhancing model interpretability and flexibility while ensuring that each classification objective is optimized independently.

III.VII Batch Inference and Final Evaluation

The trained models are then evaluated on a held-out validation dataset to assess their generalization performance. To ensure consistency, the same preprocessing pipeline—including TF-IDF vectorization, Chi-Squared feature selection, and ordinal encoding—is applied to the validation data as was used during training. Predictions are generated in batches to optimize memory usage and computational efficiency, particularly when working with large datasets. For each target variable, the following evaluation metrics are computed:

- **Top Category Prediction :** Validation accuracy and weighted F1 score to assess classification performance across potentially imbalanced classes.
- **Bottom Category Prediction:** Validation accuracy and weighted F1 score to ensure consistent predictive capability on deeper hierarchical labels.

This evaluation strategy provides a comprehensive measure of model effectiveness across all classification objectives, enabling fine-grained analysis of strengths and weaknesses across target labels.

IV Exploratory Data Analysis

A series of exploratory data analysis (EDA) techniques was applied to better understand the dataset’s distribution and characteristics before modeling. This process included analyzing text patterns within product titles, inspecting categorical feature distributions, and identifying potential data imbalances.

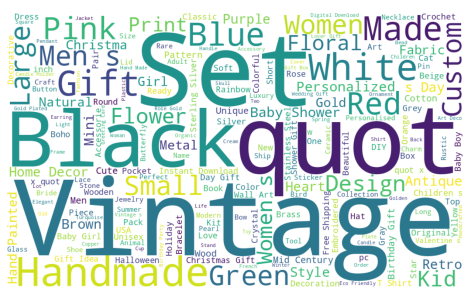


Figure 2: Word Cloud

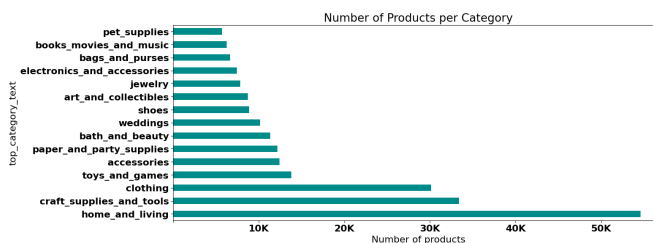


Figure 3: Products by Category Distribution

- **WordCloud on Product listings** The word cloud highlights the most frequently used terms in product listings. Common words included “custom,” “vintage,” “gift,” and “personalized,” which reflect the handcrafted and personalized nature of many items in the dataset. Domain-specific stopwords such as “free,” “new,” and “shipping” were excluded to improve the clarity of the results.

- **Distribution of Products by Category** To understand the distribution of products across various categories, we visualized the number of products listed in each top-level category using a horizontal bar chart.

From the plot, we observe that:

- Home and Living is by far the most dominant category, with over 50,000 products, indicating it's a primary focus for sellers.
- Craft Supplies and Tools and Clothing follow with around 33,000 and 30,000 products respectively, showing a high concentration of listings in DIY and fashion-related items.
- Categories like Toys and Games, Accessories, and Paper and Party Supplies form the mid-range group, with product counts between 12,000 to 15,000. On the lower end, categories such as Pet Supplies, Books, Movies, and Music, and Bags and Purses each have fewer than 7,000 products.

This skewed distribution suggests a marketplace trend where sellers and buyers are more active in home-related, crafting, and clothing niches. It also points toward potential opportunities for growth in underrepresented categories.

- **Distribution of Products per Top Category ID :** The bar chart titled "Distribution of Bottom Category ID" (by top category) shows the frequency of product listings across various top-level category IDs. Each ID corresponds to a different top-level product category.

Observations: Top Category ID 8 has the highest number of listings (over 50,000), indicating it's a dominant area in the dataset. Top Category IDs 5 and 6 also have significantly high frequencies (around 30,000–35,000), making them important categories. Several categories, such as IDs 4 and 11, have lower product counts (under 10,000), suggesting less representation or niche focus.

The chart highlights a non-uniform distribution, where a few categories dominate the product space. This imbalance can have implications for model training or market analysis, as models may be biased toward more represented categories unless techniques like class balancing or reweighting are applied.

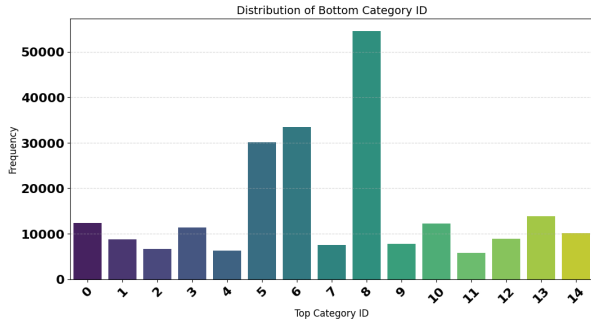


Figure 4: Distribution of Products per Top Category ID

- **– Distribution of Bottom Category Frequencies :** The second plot is a histogram of the frequency of products across all 2900 bottom-level categories, overlaid with a KDE (Kernel Density Estimation) curve.

Observations: Most bottom-level categories have frequencies clustered between 85 to 95, suggesting a well-distributed set for many classes. There are a few categories with frequencies as low as 40, but these are rare. The shape of the distribution is right-skewed, with a concentration around the upper-middle frequency range, tapering off towards the higher and lower ends. This chart provides reassurance that, at the most granular level, the dataset maintains a relatively balanced frequency across the bottom categories, which is favorable for tasks like fine-grained classification or recommender systems.

V Results

In this study, I followed a systematic approach for processing and analyzing the dataset. The methodology involved several key steps: text preprocessing

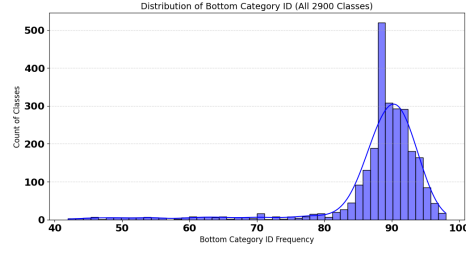


Figure 5: Bottom Category Frequencies

using Natural Language Processing (NLP), feature selection through the Chi-Square (ChiSq) test, ordinal encoding for categorical data transformation, followed by feature selection with the Best k feature selection technique, and finally, the use of Logistic Regression (LR) and Random Forest (RF) models for classification.

- **Model Performance** The classification models were evaluated based on standard metrics such as accuracy, precision, recall, and F1-score. These metrics provide insights into the model's ability to correctly classify data, balancing false positives and negatives. Below, we present the overall evaluation results and provide a detailed breakdown for each category.
- **Top-level Performance: Accuracy: 80.50% and F1-Score: 80.14%** This indicates that the models achieved a good overall performance, with an ability to correctly classify approximately 80% of the data, and the F1-score further reflects a strong balance between precision and recall across all categories.
- **Detailed Classification Metrics** The table below presents the performance metrics for each individual category. These metrics indicate how well the model performed across different classes, with an emphasis on precision, recall, and F1-score.
- **Category 5** achieved the highest performance, with a precision of 0.90, a recall of 0.91, and an F1-score of 0.90. This indicates that the model was able to correctly classify the majority of instances for this category, with minimal false positives and false negatives. The high recall suggests that it effectively captures most true instances from this category.
- **Category 8**, on the other hand, presented the opposite trend with a high recall of 0.91 but a lower precision of 0.73. This suggests that while the model correctly identifies most instances from this category, it also includes a significant number of false positives. The F1-score of 0.81 reflects a decent trade-off between

Category	Precision	Recall	F1-Score	Support
0	0.76	0.70	0.73	1243
1	0.83	0.52	0.64	870
2	0.81	0.70	0.75	664
3	0.83	0.75	0.79	1140
4	0.82	0.73	0.77	626
5	0.90	0.91	0.90	3004
6	0.83	0.85	0.84	3337
7	0.87	0.76	0.81	747
8	0.73	0.91	0.81	5460
9	0.85	0.85	0.85	782
10	0.79	0.75	0.77	1223
11	0.89	0.79	0.83	574
12	0.92	0.91	0.92	891
13	0.82	0.70	0.75	1387
14	0.74	0.49	0.59	1015

Table 1: Validation Results

precision and recall, indicating that the category was relatively well handled despite some imbalance.

- **Category 1** had the lowest performance in terms of recall (0.52), although its precision was high at 0.83. This indicates that while the model is good at identifying instances that belong to this category, it misses many true instances, leading to a lower recall and F1-score of 0.64. This could suggest a need for further model optimization or feature selection for better prediction accuracy in this category.
- Categories such as 6 (precision of 0.83, recall of 0.85, F1-Score of 0.84) and 12 (precision of 0.92, recall of 0.91, F1-Score of 0.92) demonstrate strong overall performance with balanced precision and recall. These categories show that the model is robust in predicting instances with high accuracy.

VI Conclusion

This study presents a comprehensive approach to solving a multi-class classification problem using a combination of Natural Language Processing (NLP) techniques and machine learning models, specifically Logistic Regression (LR) and Random Forest (RF). By incorporating a well-structured pipeline that includes text preprocessing, feature selection through the Chi-Square test, ordinal encoding for categorical data, and feature reduction using the Best k feature selection method, we were able to achieve a robust classification model that performs effectively across a wide range of categories.

The model achieved an overall accuracy of 80.50% and an F1-score of 80.14%, demonstrating a strong

ability to classify the data correctly while maintaining a good balance between precision and recall. The weighted average F1-score of 0.80 indicates that the model is adept at handling both the dominant and minority classes without significant bias towards either, which is crucial for real-world applications where data imbalance is often a challenge.

References

- [1] Y. Ibrahim, S. M. Yusuf, E. Okafor, Z. M. Abubakar, B. Yahaya, and U. Y. Bagaye, "Comparative Study of Ensemble Learning Techniques for Text Classification," Proc. Int. Conf. Multidisciplinary Engineering and Applied Science (ICMEAS), 2021.
- [2] Y.-S. Dong and K.-S. Han, "A Comparison of Several Ensemble Methods for Text Categorization," Proc. IEEE Int. Conf. Services Computing (SCC), 2004, pp. 419–426.
- [3] R. Patil, S. Boit, V. Gudivada, and J. Nandigam, "A survey of text representation and embedding techniques in NLP," IEEE Access, vol. 11, pp. 12345–12356, Apr. 2023.
- [4] N. Ranjan, K. Mundada, K. Phaltane, and S. Ahmad, "A survey on techniques in NLP," Int. J. Comput. Appl., vol. 134, no. 8, pp. 6–10, Jan. 2016.
- [5] D. M. Powers, "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness Correlation," J. Machine Learning Technologies, vol. 2, no. 1, pp. 37–63, 2011.